

PulseRank Platform

v1.0

Production-simulated marketplace recommendation and ranking decision system
with IPS bias correction, delayed attribution, exposure governance,
offline A/B simulation, and evidence artifacts.

Metric	Value
Sessions	4,132
Items / Sellers	650 / 80
Impressions	41,320
Purchases	539
Hybrid Recall@100	0.690
Holdout NDCG@10 (biased)	0.134
IPS-weighted NDCG@10	0.522
Seller Gini after rerank	0.582
Catalog coverage@10	0.226
Offline A/B decision	HOLD_SIMULATED
JSON evidence artifacts	33
Failure scenarios	15

1. Executive Summary

PulseRank is a production-simulated marketplace ranking decision system. It demonstrates the full lifecycle of a marketplace recommendation platform: from corpus generation and candidate retrieval through ranking evaluation, position-bias correction, exposure governance, delayed attribution, and offline A/B decision-making.

The system is built as a solo portfolio project with deliberate production simulation — every claim is backed by an executable script and a JSON artifact. The A/B decision is labeled `HOLD_SIMULATED` to distinguish it explicitly from a live experiment result.

1.1 Key Design Decisions

Display-rank logging at impression time. Every impression records the position at which the item was shown. Without this, IPS debiasing is impossible. This is the foundational observability requirement that most recommendation notebooks omit.

Temporal train/test split. The holdout is strictly time-ordered. No future purchase labels appear in training features. Temporal leakage is the single most common offline evaluation failure in recommendation systems.

IPS before NDCG. Naive $\text{NDCG}@10 = 0.134$ understates quality because position-1 items get far more clicks than position-10 items regardless of relevance. IPS reweights each observation by the inverse of its propensity. $\text{IPS NDCG}@10 = 0.522$.

HOLD_SIMULATED A/B decision. The offline A/B simulation does not show sufficient lift to justify launch. Reporting a HOLD decision is more rigorous than reporting any improvement as a win. A reviewer who sees only wins should be suspicious.

Truth boundary: PulseRank is non-production. It does not claim real users, real traffic, real revenue optimization, or real online A/B testing.

2. System Architecture

PulseRank is organized as eight build groups, each producing verifiable JSON artifacts. The pipeline flows from corpus generation through candidate retrieval, ranking, bias correction, reranking, attribution, and A/B simulation.

2.1 Build Groups

Group	Module	Key Output
0 — Foundation	Repo scaffold	Schema manifest, config
1 — Corpus	core/corpus.py	corpus_summary.json, impression_log with display_rank

2 — Candidate Gen	core/candidates.py	candidate_generation_report.json, Recall@100=0.690
3 — Ranking Baseline	ranking/baseline.py	ranking_baseline_report.json, NDCG@10=0.134
4 — IPS Correction	evaluation/ips.py	bias_correction_report.json, IPS NDCG@10=0.522
5 — Reranking	ranking/reranker.py	diversity_report.json, Gini=0.582
6 — Attribution + A/B	simulation/ab.py	ab_simulation_results.json, HOLD_SIMULATED
7 — Failure Scenarios	artifacts/failures.py	failure_recovery_report.json, 15 scenarios
8 — Showcase	docs/, dashboard	GitHub Pages dashboard, 33 artifacts

2.2 Data Flow

The 90-day synthetic corpus is generated deterministically from a fixed seed. Sessions are assigned to users with realistic browsing patterns. Items have feature vectors, categories, and seller assignments. Impressions are logged with explicit display_rank at serve time — this is the critical observability hook.

Candidate generation produces a pool of 100 items per session using a popularity scorer and a content-based scorer. These are merged into a hybrid ranked list. The baseline ranker then applies a temporal holdout evaluation on this pool.

IPS bias correction estimates the propensity of each display rank position using a position-based click model. Items shown at position 1 are far more likely to be clicked than items at position 10, regardless of relevance. IPS weights invert this bias to produce a debiased evaluation metric.

3. Component Deep Dive

3.1 Corpus Generation

4,132 user sessions are generated over a 90-day window with 650 items across 80 sellers. Each session contains a browsing sequence with impressions at ranked positions 1–10. The impression log records: session_id, item_id, display_rank, timestamp, and whether a purchase occurred within a 30-day attribution window.

Display-rank coverage = 1.0 means every impression in the corpus has a logged display rank. This is a deliberate design requirement — without it, propensity estimation is impossible.

3.2 Candidate Generation

Generator	Signal	Recall@100
Popularity scorer	Global purchase rate per item	~0.62
Content-based scorer	Item feature cosine similarity	~0.58
Hybrid merge	Weighted combination	0.690

Hybrid Recall@100 = 0.690 means that for 69% of sessions with a purchase, the purchased item appears in the top-100 candidates. Items outside this pool cannot be ranked or shown, so recall is the hard ceiling on ranking quality.

3.3 Ranking Baseline and Temporal Evaluation

The baseline ranker uses a supervised model trained on impression features. The train/test split is strictly temporal: the last 20% of sessions by time form the holdout. This prevents any leakage of future purchase outcomes into training.

Holdout NDCG@10 = 0.134 using naive click labels. This is the position-biased metric that overstates the difficulty of the ranking task because high-rank impressions are more likely to be clicked by position alone.

3.4 IPS Position-Bias Correction

A position-based click model estimates $p(\text{click} \mid \text{rank}=k)$ for each display rank k . In PulseRank, the propensity model uses a power-law decay: items at rank 1 have propensity 1.0, and propensity decays with position depth.

IPS weight for each observation: $w = 1 / \text{propensity}(\text{display_rank})$. Weights are clipped at a maximum value to control variance from very low propensity items.

The debiased NDCG@10 = 0.522 vs naive 0.134. The large difference shows how severely position bias inflates the apparent difficulty of ranking when using raw click labels as ground truth.

Key interview point: IPS does not improve the ranker. It improves the measurement of the ranker. The model is the same — the evaluation is more honest.

3.5 Reranking with Exposure Constraints

Constraint	Purpose	Result
MMR diversity	Reduce redundancy in top-10	Novelty score improved
Seller Gini	Prevent seller concentration	Gini after rerank = 0.582
Catalog coverage@10	Ensure category breadth	Coverage = 0.226
Novelty filter	Surface less-seen items	Applied post-MMR

The Gini coefficient of 0.582 indicates moderate seller concentration in the reranked results — a Gini of 1.0 would mean one seller captures all slots, while 0.0 would mean perfect equality. The governance constraint prevents the pure-relevance ranker from concentrating results in the top few sellers.

3.6 Delayed Conversion Attribution

Purchase events are attributed to the last relevant impression within a 30-day window. This models the realistic delay between browsing and purchase — a user who views an item today may purchase it two weeks later after a salary cycle.

The `conversion_attribution_report.json` documents attribution window analysis: what fraction of purchases were attributed within 1 day, 7 days, and 30 days. Long-tail attribution is a commonly missed complexity in recommendation evaluation.

3.7 Offline A/B Simulation

Control condition: baseline ranker with no IPS correction. Treatment condition: IPS-corrected ranker with reranking constraints. The simulation replays held-out sessions through both conditions and computes per-session metric differences.

Decision: `HOLD_SIMULATED`. The treatment shows improvement in IPS NDCG@10 but does not cross the minimum detectable effect threshold for a launch recommendation. A `HOLD` is more honest than reporting any positive delta as a win.

3.8 MetaSignal Integration

PulseRank emits 15 structured events in a MetaSignal-compatible format. These events cover: ranking model updates, IPS weight recalculation, governance constraint violations, attribution window closures, and A/B phase transitions. This demonstrates how ranking systems feed into broader experiment intelligence platforms.

4. Failure Scenarios

15 failure scenarios are documented with recovery paths. Selected scenarios:

Scenario	Failure Mode	Recovery
Cold-start item	New item has no purchase history	Content-based fallback, boosted exploration
Cold-start user	New session, no history	Global popularity baseline
Propensity collapse	All items estimated same propensity	Fallback to position-based prior
Gini constraint breach	Single seller dominates candidates	Hard cap per seller in reranker
Attribution timeout	Purchase after 30-day window	Logged as unattributed, excluded from training
Recall collapse	Candidate pool misses purchased item	Flagged in evaluation log, not a ranker issue
IPS weight explosion	Very low propensity item	Clipping at configured max weight
Temporal leakage	Future labels in training features	Strict date-boundary filter in pipeline

5. Evidence Artifacts

All 33 JSON artifacts are generated by executable scripts in scripts/. Each artifact is versioned and reproducible from a fixed seed.

Artifact	What it proves
corpus_summary.json	Corpus statistics, display-rank coverage = 1.0
candidate_generation_report.json	Hybrid Recall@100 = 0.690
ranking_baseline_report.json	NDCG@10, MRR, hit rate, temporal split
offline_eval_log.json	Per-session evaluation with no leakage
model_registry.json	Versioned model + config snapshot
bias_correction_report.json	IPS before/after: 0.134 vs 0.522
propensity_by_rank_report.json	Propensity decay by display rank
diversity_report.json	MMR, Gini, novelty, coverage metrics
diversity_guardrail_log.json	Per-session constraint application log
catalog_coverage_report.json	Coverage@k across catalog
conversion_attribution_report.json	Attribution window analysis
ab_simulation_results.json	Control vs treatment, HOLD_SIMULATED
metasignal_integration_events.json	15 structured observability events
failure_recovery_report.json	15 failure scenarios with recovery paths

6. Strength Assessment

Dimension	Assessment	Score
Architecture depth	8 build groups, full pipeline from corpus to A/B decision	Strong
IPS implementation	Propensity estimation, weight clipping, before/after eval	Strong
Evaluation rigor	Temporal split, no leakage, debiased metric	Strong
Exposure governance	Gini, MMR, coverage — all three constraints wired	Strong
A/B simulation	HOLD decision more honest than cherry-picked win	Strong
Evidence quality	33 JSON artifacts, all executable and reproducible	Strong
Failure scenarios	15 scenarios, realistic failure modes covered	Good
Truth boundary	Explicitly non-production, no overclaims	Good
MetaSignal integration	15 events, shows cross-system awareness	Good
Online evaluation	No real A/B test — simulation only	Acknowledged gap
RL / bandit layer	Not implemented — pure offline system	Acknowledged gap

The acknowledged gaps are appropriate. A production system would eventually add online A/B infrastructure and potentially a bandit layer for exploration. For a portfolio project, the offline simulation with an honest HOLD decision is more credible than a simulated online win.

7. Anticipated Interview Questions

Why is IPS NDCG@10 so much higher than naive NDCG@10?

Naive NDCG uses raw click labels. Position 1 gets far more clicks than position 10 regardless of item quality — this is position bias. IPS reweights each click by $1/\text{propensity}(\text{rank})$, so clicks at low positions count more because they are rarer and therefore more informative about true relevance. The 4x improvement (0.134 to 0.522) is not a model improvement — it is the correction of a measurement error.

Why HOLD_SIMULATED and not a win?

Because the treatment does not cross the minimum detectable effect threshold in offline simulation. Reporting any positive delta as a win is a common portfolio mistake. A HOLD is more rigorous and more believable to a senior reviewer who has seen too many offline wins that failed online.

What does display-rank coverage = 1.0 mean?

Every impression in the corpus has a logged display rank. This is a data quality requirement for IPS, not a model quality metric. In production, display ranks can be missing due to logging failures, A/B holdout groups, or feature flag issues. Coverage = 1.0 here means the synthetic corpus was generated with perfect logging.

Why does Recall@100 matter more than NDCG@100?

Recall@100 is the ceiling on ranking quality. Items outside the candidate pool cannot be ranked into the top-10 no matter how good the ranker is. If Recall@100 is 0.69, then 31% of sessions have a purchased item that the ranker never sees. Improving candidate generation (raising this recall) is often a higher-ROI investment than improving the ranker for a fixed candidate pool.

What is the seller Gini and why does it matter?

The seller Gini coefficient measures concentration of ranking slots across sellers. A pure relevance ranker tends to concentrate results in the most popular sellers, which creates a rich-get-richer dynamic that harms marketplace health. The Gini constraint enforces a maximum concentration, trading a small amount of relevance for fairer seller exposure. After reranking, Gini = 0.582.

How does delayed attribution work?

A purchase within 30 days of an impression is attributed to that impression's item. The attribution is last-touch within the window. This matters because: (1) using same-session clicks as the only signal misses long-consideration purchases, (2) attribution window length is a hyperparameter that affects training label quality, and (3) purchases outside the window are unattributed and excluded from training to avoid negative label noise.

What would v2 of PulseRank add?

Online A/B infrastructure — even simulated using a multi-armed bandit replay. Real-time propensity recalibration. LTR (learning to rank) model replacing the baseline ranker. Stream-based attribution using Kafka-compatible event logs. The MetaSignal integration already provides the observability scaffolding for these extensions.

8. Truth Boundary

PulseRank is a solo-built, non-production, production-simulated system.

It does **not** claim:

- Real production deployment or real users served
- Real online A/B testing or real revenue optimization
- Real RL, contextual bandits, or ad-auction infrastructure
- Real streaming or real-time serving
- Production SLAs or latency guarantees

Every major claim is backed by an executable script in `scripts/` and a corresponding JSON artifact in `outputs/`. The offline A/B decision is labeled `HOLD_SIMULATED`. The corpus is synthetic and deterministic from a fixed seed.

Resume-Safe Claim

Built PulseRank, a production-simulated marketplace ranking system with display-rank impression logging, hybrid candidate generation ($\text{Recall@100}=0.690$), temporal holdout ranking evaluation, IPS position-bias correction (NDCG@10 from 0.134 to 0.522 debiased), delayed conversion attribution, seller/category exposure governance, offline A/B simulation (`HOLD_SIMULATED`), 15 failure recovery scenarios, MetaSignal-compatible event emission, and a GitHub Pages evidence dashboard covering 33 JSON artifacts.